

User Guide

One workspace across Claude Code and Codex, on your Mac, your terminal and your iPhone.

Version 0.1 · November 2026

Contents

1 What WotsRelay does

2 Before you start

3 Install it once

4 Your first session

5 Working in a session

6 Models and automatic switching

7 When a reply pauses

8 The desktop app

9 The iPhone app

10 Command reference

11 Privacy and safety

12 Troubleshooting

13 What’s coming next

1 · What WotsRelay does

WotsRelay lets you work with Claude and Codex as if they were one assistant. You talk to it like you would to `claude` or `codex` in a terminal, and when one provider is about to run out of its usage limit, the conversation continues on the other one, with everything it needs to pick up where it left off.

- **Claude Opus 5.5 is your default model.** GPT-6 Astra (through Codex) is the fallback. You can pick any other Claude or Codex model at any time.
- **It switches before you hit the wall.** When a provider reports that 95% of its 5-hour or weekly limit is used, your next message goes to the other model, and switches back once the limit resets.
- **Nothing is lost when it switches.** The next model receives the conversation so far, the files that changed and what was already done, so it doesn't repeat work.
- **Work keeps going on your Mac.** A small background service (the *runner*) does the work. The terminal, the desktop app and the iPhone app are just windows onto it.
- **It uses your existing subscriptions.** WotsRelay drives the official `claude` and `codex` tools you're already signed into. It never asks for or stores your passwords.

> **Every reply is labelled.**
> Each answer shows which model wrote it, so you always know whether Claude or Codex is speaking.

2 · Before you start

YOU NEED	CHECK WITH
A Mac with Apple silicon	—
Node.js 26 or newer and pnpm 11	`node --version` · `pnpm --version`
Claude Code, signed in	`claude auth status` (sign in with `claude auth login`)
Codex CLI, signed in	`codex login status` (sign in with `codex login`)
For the iPhone app: Xcode	Only needed to build the app yourself

Sign in with the official tools.
WotsRelay never handles your Claude or ChatGPT passwords. If either tool says you're signed out, sign in with its own command, then come back.

3 · Install it once

1. **Get the code and install it.**

```
cd ~/dev
git clone https://github.com/lyndonjugalbot-cpu/WotsRelay.git WotsAI
cd WotsAI
pnpm install
```

2. **Put `wots` on your PATH** so you can run it from any folder. Run this from inside the project folder:

```
mkdir -p ~/.local/bin && ln -sfn "$(pwd)/apps/cli/bin/wots" ~/.local/bin/wots
```

If your terminal then says `command not found: wots`, add `~/.local/bin` to your PATH once:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc && source ~/.zshrc
```

3. **Check that everything is ready.**

```
$ wots doctor
● claude-code  2.1.316  auth=authenticated  billing=subscription
● codex  0.166.0  auth=authenticated  billing=subscription
● no billing-changing environment variables detected
```

Both providers should say `auth=authenticated` and `billing=subscription`.

> **Billing check.**
> If `wots doctor` warns about variables such as `ANTHROPIC_API_KEY` or `OPENAI_API_KEY`, those would make the tools bill an API account instead of your subscription. WotsRelay refuses to run a provider while such a variable is set, unless you deliberately allow API billing.

4 · Your first session

Open a terminal **inside the project folder you want to work on** and type:

```
$ cd ~/dev/my-project
$ wots
```

The first time in a folder, WotsRelay asks whether you trust it. Use the arrow keys and Enter:

```
Do you trust the files in this folder?
  ~/dev/my-project
  Claude and Codex will be able to read and edit files and run commands here.
❯ 1. Yes, trust this folder
  2. No, chat only (no file access)
```

- **Yes** lets the models read and edit files and run commands in that folder (risky commands still ask you first).
- **No, chat only** gives you a conversation with no file access at all.
- In your home folder WotsRelay always uses chat only. Run it inside a project folder to work on code.

Then you see the session banner and a prompt:

```
✻ WotsRelay  Claude Opus 5.5  → fallback GPT-6-Astra
  ~/dev/my-project  edits allowed
  /model to switch models · /help for commands · Ctrl-C stops a reply

❯ Add a dark mode toggle to the settings page
```

Type what you want and press Enter. The reply streams in, labelled with the model that wrote it, and you can keep the conversation going: follow-up messages remember everything said so far.

> **Start with a request directly.**
> `wots "fix the failing test"` opens the session and sends that message straight away. Piped input runs once and exits: `echo "summarize README.md" | wots`.

5 · Working in a session

Commands inside a session

TYPE	WHAT IT DOES
`/model`	Pick any Claude or Codex model with the arrow keys. The other provider stays as the fallback, and your choice is saved as the default.
`/model gpt-6-astra`	Switch directly to a named model.
`/new` (or `/clear`)	Start a fresh conversation.
`/resume`	Pick an earlier conversation in this folder and continue it.
`/status`	Current model, fallback, folder access, sign-in, and usage bars for each provider.
`/help`	List these commands.
`/exit`	Quit. Ctrl-D, or Ctrl-C twice on an empty prompt, also quits.

Keys

KEY	WHAT IT DOES
Ctrl-C while a reply runs	Stops that reply. You stay in the session.
Ctrl-C on an empty prompt	Shows “press Ctrl-C again to exit”; press it again to quit.
↑ / ↓ at the prompt	Recall earlier messages.

Approvals

When a model wants to do something that needs your permission, WotsRelay shows it in plain words and asks:

```
? Approval needed  reaches outside this folder (network, deletion or system change)
  Run: curl -sI https://example.com
  in ~/dev/my-project
Allow this?
› 1. Approve once
  2. Approve for this session   similar requests run without asking until the session ends
  3. Reject                      Esc also rejects
```

RISK SHOWN	MEANING
only reads	Looks at files or status without changing anything.
changes files in this folder	Creates or edits files inside your project.
reaches outside this folder	Network access, deleting files, installing packages, pushing to git or changing the system.
not recognised	WotsRelay can’t tell what the command does. Read it before approving.

“Approve for this session” is offered only when the provider supports it (Codex commands).

6 · Models and automatic switching

Choosing a model

Type `/model` in a session. Claude models: Opus 5.5 (default), Sonnet 5.5, Haiku 5.5. Codex models: the list your ChatGPT plan offers, including GPT-6 Astra. The model you pick goes first; the other provider stays as the fallback.

Switching at 95%

Claude and Codex both report how much of their **5-hour** and **weekly** limits you've used. WotsRelay watches those numbers:

- From **80%** you see a short heads-up after a reply, for example *"Claude Opus 5.5: 86% of the 5-hour limit used (resets 18:56). Switches to GPT-6 Astra at 95%."*
- At **95%**, your **next message** goes to the other model, with the whole conversation. A reply that's already running is never cut off.
- When the limit **resets**, the next message goes back to your preferred model automatically.

```
⇄ claude-opus-5-5 is at 96% of its 5-hour limit, resets 06:56 PM. Continuing with gpt-6-astra on codex.
```

See and change it

COMMAND	WHAT IT DOES
`/status` or `wots usage`	Usage per provider and window, with reset times.
`wots fallback threshold 90`	Switch at 90% instead of 95% (allowed range 50–100).
`wots fallback show`	Current order, effort and whether automatic fallback is on.
`wots fallback set claude-opus-5-5 gpt-6-astra`	Set the order of models directly.

> **Your permission comes first.**
> Moving a conversation to the other provider shares it with that provider. The first time that's needed, WotsRelay asks: *"Send this conversation to GPT-6 Astra to continue?"* Choose "Yes, and allow it for this folder from now on", or "No" to stay where you are.

7 · When a reply pauses

Sometimes a reply can’t continue on its own, for example when a provider actually hits its limit, loses its connection, or needs you to sign in again. Instead of failing, WotsRelay saves a checkpoint and asks what to do:

```
The reply is paused. What next?
› 1. Continue with GPT-6 Astra   picks up from the saved checkpoint
  2. Retry with Claude Opus 5.5
  3. Leave it paused              type a new message or /resume later
```

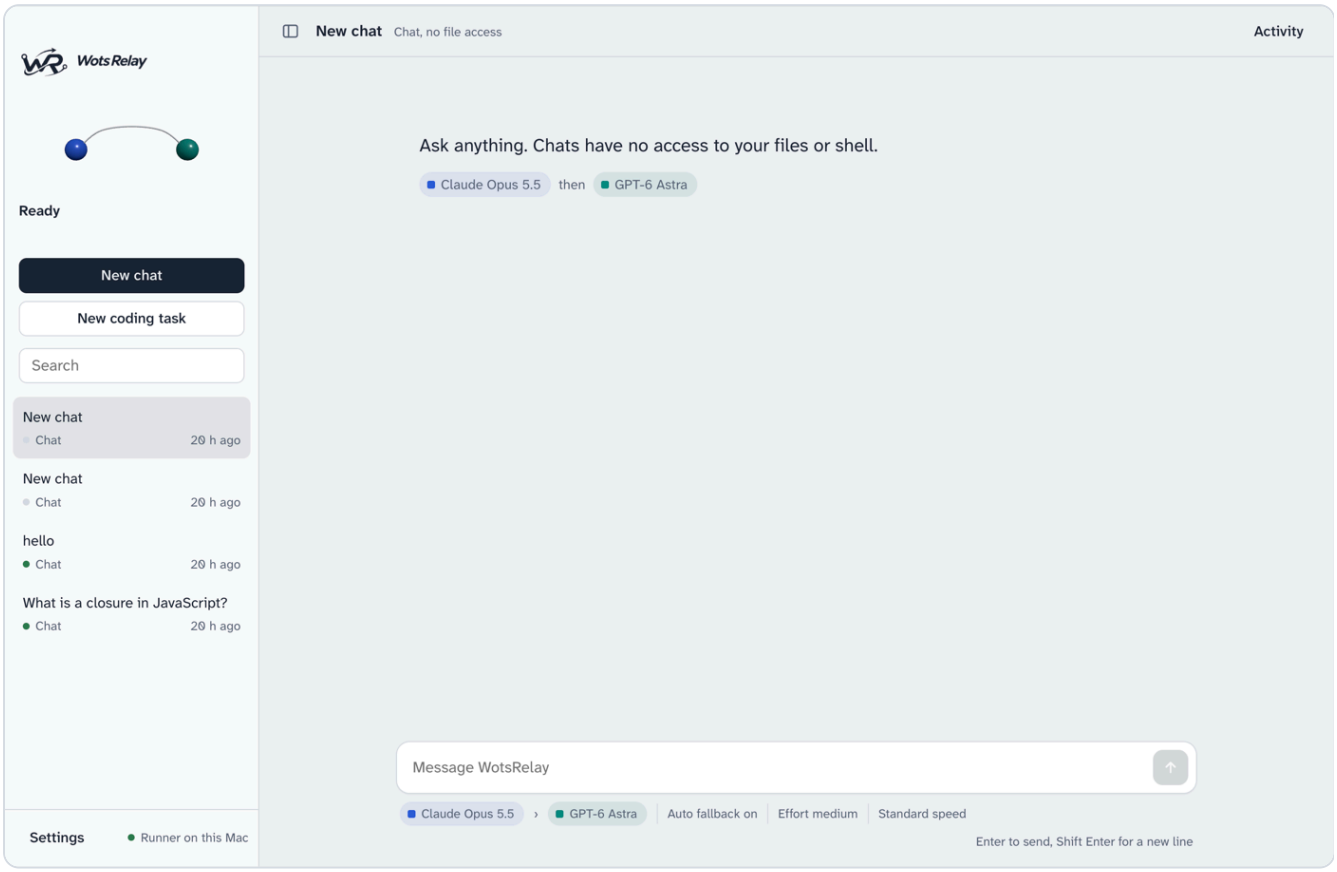
- **Continue with …** hands the task to the other model. It receives the goal, your latest instructions, what was completed, the files that changed and their diff, and is told not to repeat finished work.
- **Retry** tries the same model again (for example after you’ve signed in again in another terminal).
- **Leave it paused** keeps everything; you can come back later with `/resume`, from the desktop app or from your iPhone.

> **If a command’s outcome is unknown,**
> such as a deploy that may or may not have run, WotsRelay pauses rather than letting the next model run it again. Check the result yourself, then continue.

8 · The desktop app

The desktop app shows the same sessions as the terminal, with the relay status, a conversation view and an activity panel. It is currently a development build that you start from the project folder:

```
$ pnpm desktop:build
$ env -u ELECTRON_RUN_AS_NODE pnpm --filter @wots/desktop start
```



The desktop app: sessions on the left, the relay status under the logo, the conversation and the model order at the bottom.

- Closing the window doesn't stop running work. The runner keeps going in the background.
- The logo's arrow sweeps again every time work is handed from one model to the other.
- Settings (⌘,) shows provider sign-in, the fallback order and simulated demos you can try without using any quota.

9 · The iPhone app

The iPhone app lets you follow sessions, answer approvals and continue paused work while you're away from your desk. The work itself still runs on your Mac, so the Mac must be awake.

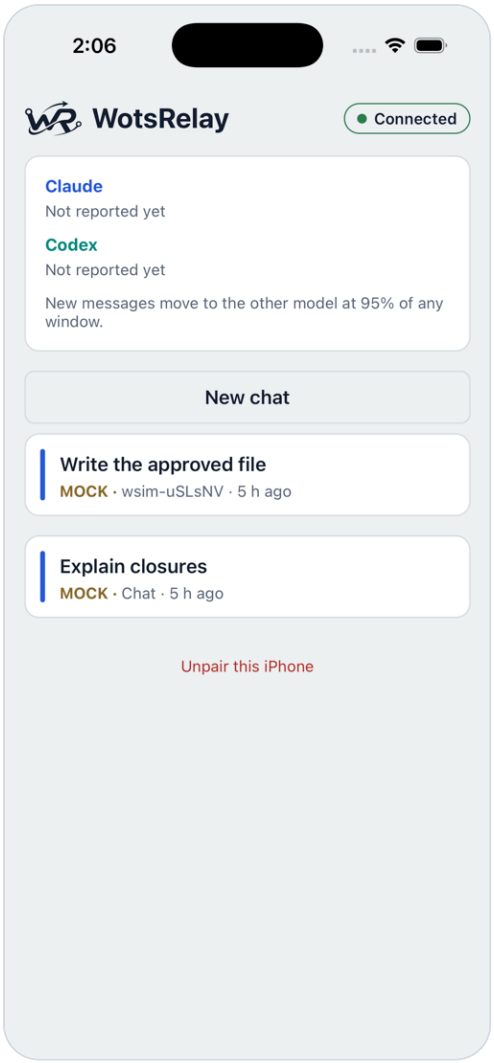
Build it (one time)

```
$ cd apps/mobile && npm install
$ npx expo run:ios        # builds and opens it in the iOS Simulator
```

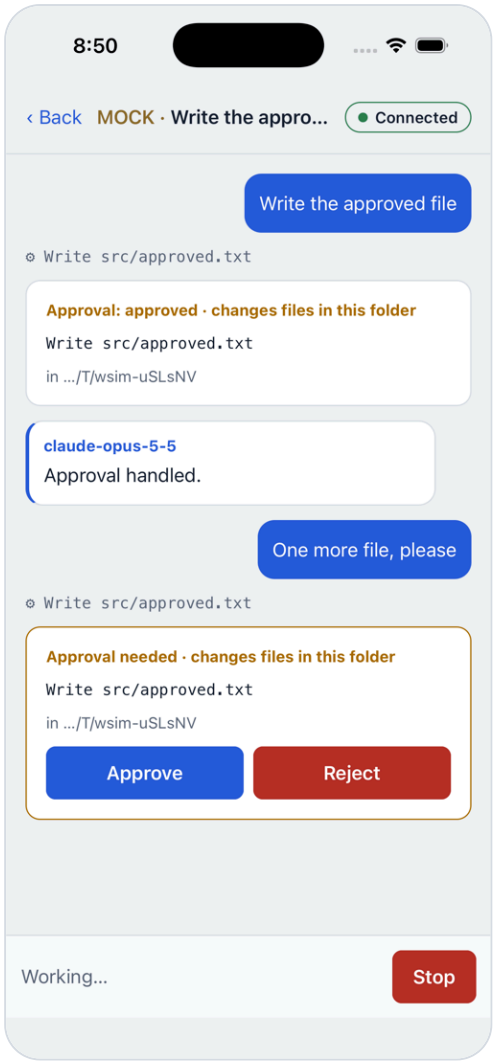
Installing it on a physical iPhone needs your Apple developer signing set up in Xcode.

Pair it with your Mac

1. On the Mac, turn on phone access: `wots gateway enable`
2. Create a pairing link: `wots gateway pair`. It's valid for 2 minutes and works once.
3. Open the link on the iPhone (AirDrop, Messages or Notes), or paste it into the app. It pairs automatically.



Session list with usage bars. Sessions waiting for you come first.



An approval arriving live, with Approve / Reject and Stop.



The app on the home screen.

Remove a phone

```
$ wots devices list
$ wots devices revoke dev_…      # disconnects it immediately
$ wots gateway disable           # turn phone access off entirely
```

> **Your phone only ever talks to your Mac.**
> The pairing link carries your Mac's certificate fingerprint; the app refuses any other server. The phone can follow and steer sessions but can't change settings, register projects or run commands of its own. To reach your Mac away from home, use your own VPN (for example Tailscale) and enable the gateway on that address.

10 · Command reference

Everything you can do outside an interactive session. Add `--json` before a command for machine-readable output, for example `wots --json sessions`.

COMMAND	WHAT IT DOES
`wots`	Interactive session in the current folder.
`wots doctor`	Tool versions, sign-in, billing route and runner state.
`wots setup`	Guided check, plus permission to continue chats on the other provider.
`wots chat "…"`	One chat message with no file access.
`wots code --project <path> "…"`	One coding task in a project folder. Add `--consent` to allow handing it to the other provider.
`wots sessions`	List sessions.
`wots resume <id> [--fallback]`	Resume a paused session, optionally on the next model.
`wots approve <id> --version <n> [--session]`	Approve a pending action (`wots reject …` rejects).
`wots models` · `wots connections`	Known models; provider connection status.
`wots usage`	Usage per provider and window.
`wots fallback show | set | threshold`	See or change the model order and the switch point.
`wots export <id> [--format json]`	Export a session as Markdown or JSON (secrets redacted).
`wots gateway enable | pair | status | disable`	Phone access.
`wots devices list | revoke <id>`	Paired phones.
`wots runner status | start | stop`	The background runner.

Exit codes for scripts: 0 completed, 10 paused, 1 failed, 130 cancelled, 3 runner unavailable.

11 · Privacy and safety

- **Everything stays on your Mac** in `~/Library/Application Support/WotsRelay`, readable only by your user account.
- **No passwords, no tokens.** Sign-in happens in the official `claude` and `codex` tools. WotsRelay never reads their credential files.
- **Secrets are kept out of handoffs.** Files like `.env`, keys and certificates are never sent to the other provider, and known secret formats (API keys, tokens, private keys) are blanked out of anything that is.
- **Your uncommitted changes are safe.** WotsRelay never resets, stashes, commits or pushes for you.
- **Risky actions ask first,** and the request shows exactly what will run and where.
- **Provider switching follows each provider's rules.** WotsRelay uses only your own signed-in accounts and never rotates accounts or works around limits.

12 · Troubleshooting

PROBLEM	WHAT TO DO
`command not found: wots`	Repeat step 2 of the installation from inside the project folder, then run `rehash` or open a new terminal tab.
A provider shows as signed out	Sign in with `claude auth login` or `codex login`, then choose **Retry** in the pause menu.
“Runner did not start”	Run `wots runner status`; the message points to `logs/runner.log`. A very long custom `WOTS_HOME` path can be too long for macOS sockets.
The phone says “Reconnecting…”	Check the Mac is awake and on the same network (or VPN), and that `wots gateway status` shows it listening.
“This pairing link was already used or has expired”	Links last 2 minutes and work once. Run `wots gateway pair` again.
The phone says it was unpaired	It was revoked on the Mac. Pair it again with a new link.
The model didn’t switch at 95%	It switches on your *next* message, never mid-reply. If you haven’t allowed sharing the conversation yet, WotsRelay asks first.
Something looks wrong	`wots doctor` shows versions, sign-in and billing without revealing any secrets.

13 · What’s coming next

These are being built now and are **not available yet**:

FEATURE	STATUS
Cursor as a third provider	in progress The connection to Cursor’s command-line tool is written and tested against its published format; it becomes selectable once the provider settings commands land and it has been checked with a real Cursor account.
Gemini as a fourth provider	in progress Through Google’s official Gemini API with your own API key (billed by Google per use, separately from any Gemini subscription).
Antigravity	in progress Detection and a button to open it in its own Terminal window. Google’s terms don’t allow other apps to drive it, so WotsRelay will never automate it.
Provider settings page and `wots providers`	in progress Enable, disable and reorder providers; inspect each handoff.
QR pairing and notifications on the iPhone	planned
A signed, installable Mac app	planned Needs Apple developer signing.